Tema 3: Procesamiento de streams: Apache Spark Streaming

Ángel Miguel García Vico
Rafael Pastor Vargas

amgarcia@scc.uned.es
rpastor@scc.uned.es

Módulo 3: Gestión de datos en tiempo real

Infraestructuras Computaciones para Procesamiento de Datos Masivos

2 de diciembre de 2022

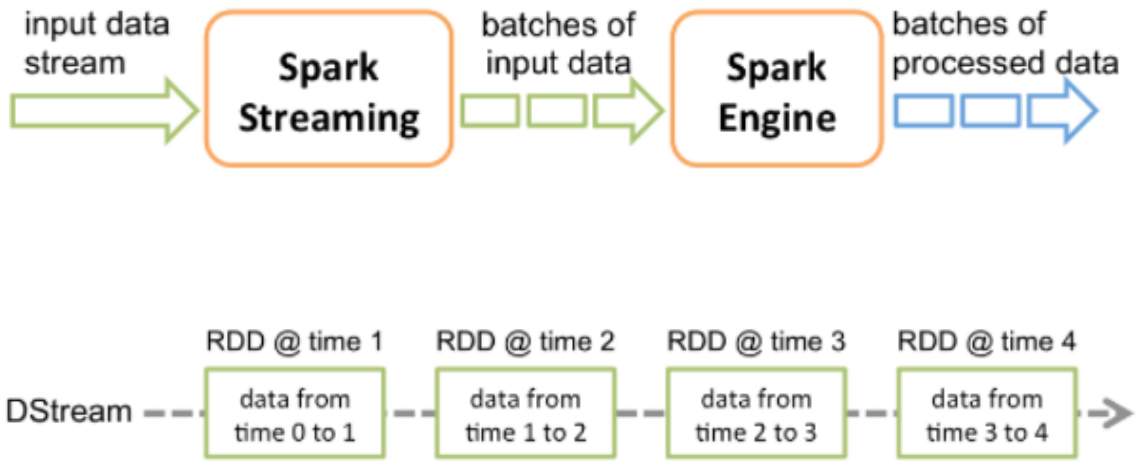


Introducción

Spark Streaming es el sistema de procesamiento masivo de flujos de datos de Apache Spark.

- Como hemos visto en el tema 1, Spark Streaming implementa una aproximación de *micro-batch* para procesar los datos de entrada en tiempo real. Es un enfoque basado en diferentes tipos de ventana de datos: se llena dicha ventana con datos del flujo y, acto seguido, se procesa.
- Es una extensión de la API de Spark: la sintaxis es prácticamente la misma.
- Se pueden leer/escribir datos desde/hacia múltiples fuentes, por ejemplo:
  - Input: Kafka, Flume, Twitter, Sockets, ...
  - Output: HDFS, BBDD, Dashboards, ...

Esquema básico



- Del flujo de entrada se genera *micro-batches*. Por defecto: **Ventanas *timestamp-based* sin solapamiento**. Se especifica el tamaño de ventana en segundos.
- Cada *micro-batch* es un `RDD` y se trabaja igual que en Apache Spark: Mismas acciones y transformaciones.
- La secuencia de *micro-batches* se almacena en un `DStream` que incluye funcionalidad adicional.
- RDD con datos de entrada $\rightarrow$ RDD con datos procesados.

Características principales

- Capa de abstracción de alto nivel. Los detalles de la implementación de streams son transparentes al usuario.
- Mismo código de Spark. Los microbatches se procesan igual que cualquier otro RDD: SparkSQL, MLlib, etc.
- Aproximación micro-batch:
  - Mayor thourghput: nº de instancias procesadas por unidad de tiempo.
  - Aumento de latencia: mínimo el tiempo de ventana del batch.
- Muy útil en arquitecturas Lambda: Spark para *offline*, Spark Streaming para *online*.
- No es lo mejor para arquitecturas Kappa.

Uso de Spark Streaming

1. Crear el *Spark Streaming Context* a partir del Spark Context, aquí se especifica la duración del micro-batch.

```
1  from pyspark import SparkContext
2  from pyspark.streaming import StreamingContext
3
4  sc = SparkContext()
5  ssc = StreamingContext(sc, batchDuration = 10)
6
```

Uso de Spark Streaming

Entrada de datos para construir el `DStream`:

- Entrada básica:
  - `fileStream()`. Para carpeta con ficheros.
  - `socketTextStream()`. Los datos provienen de un socket de una red.
  - `queueStream()`: Secuencia de RDDs de Spark, cada RDD se trata como un batch.
- Entrada avanzada:
  - Normalmente mediante librerías adicionales:
  - Kafka, Flume, Kinesis, ...
- Entrada personalizada: con conectares ad-hoc.

Uso de Spark Streaming

Ejemplo: Input a través de sockets.

```
1    from pyspark import SparkContext
2    from pyspark.streaming import StreamingContext
3
4    sc = SparkContext()
5    ssc = StreamingContext(sc, batchDuration = 10)
6
7    # Escucha desde localhost en el puerto 9999
8    dstream = ssc.socketTextStream("localhost", 9999)
9
```

Para probarlo, usar en consola el comando:
`> nc -lk 9999`
para abrir un socket en el puesto 9999 de localhost.
Introducir el texto deseado para probar (se pueden usar pipes).
Nota: Si no existe 'nc' $\rightarrow$ `sudo apt-gat install netcat`

Uso de Spark Streaming

Ejemplo: Input con Kafka.

```
1   from pyspark import SparkContext
2   from pyspark.streaming import StreamingContext
3   from pyspark.streaming.karfa import KafkaUtils
4
5   sc = SparkContext()
6   ssc = StreamingContext(sc, batchDuration=10)
7
8   dstream = KafkaUtils.createDirectStream(
9     ssc,
10    topics=['my_topic'],
11    kafkaParams={'boostrap.servers': localhost:9092})
12
```

Fijaos que la opción `boostrap.servers` es la misma que la vista en el tema 2 para la creación de un consumidor, publicar topics, etc.

Uso de Spark Streaming

Transformaciones sobre un `DStream`:

Método	Funcionamiento
map(func)	Return a new DStream by passing each element of the source DStream through a function func.
flatMap(func)	Similar to map, but each input item can be mapped to 0 or more output items.
filter(func)	Return a new DStream by selecting only the records of the source DStream on which func returns true.
repartition(numPartitions)	Changes the level of parallelism in this DStream by creating more or fewer partitions.
union(otherStream)	Return a new DStream that contains the union of the elements in the source DStream and otherDStream.
count()	Return a new DStream of single-element RDDs by counting the number of elements in each RDD of the source DStream.
reduce(func)	Return a new DStream of single-element RDDs by aggregating the elements in each RDD of the source DStream using a function func (which takes two arguments and returns one). The function should be associative and commutative so that it can be computed in parallel.
countByValue()	When called on a DStream of elements of type K, return a new DStream of (K, Long) pairs where the value of each key is its frequency in each RDD of the source DStream.

Uso de Spark Streaming

Transformaciones sobre un `DStream`:

Método	Método
reduceByKey(func, [numTasks])	When called on a DStream of (K, V) pairs, return a new DStream of (K, V) pairs where the values for each key are aggregated using the given reduce function. Note: By default, this uses Spark's default number of parallel tasks (2 for local mode, and in cluster mode the number is determined by the config property spark.default.parallelism) to do the grouping. You can pass an optional numTasks argument to set a different number of tasks.
join(otherStream, [numTasks])	When called on two DStreams of (K, V) and (K, W) pairs, return a new DStream of (K, (V, W)) pairs with all pairs of elements for each key.
cogroup(otherStream, [numTasks])	When called on a DStream of (K, V) and (K, W) pairs, return a new DStream of (K, Seq[V], Seq[W]) tuples.

Uso de Spark Straming

Salida de datos.

- `dstream.pprint()`: salida estandar.
- `dstream.saveAsTextFiles()`: Salida a almacenamiento externo.
- `dstream.foreachRDD()`: Aplica un función arbitraria a cada RDD del stream. Es el método más utilizado en Apache Spark Streaming para procesar datos.

Uso de Spark Straming

Para ejecutar el procesamiento de datos de Spark Streaming, una vez definido todo lo anterior:

- `ssc.start()`: comienza el procesamiento de datos sin bloquear el programa.
- `ssc.awaitTermination()`: el script principal queda a la espera hasta que se finalice el procesamiento del stream. Si no hacemos esto, todo el proceso acaba.

Uso de Spark Straming

Pasos a seguir para el procesamiento de datos con Spark Streaming:

- Desde consola (un script):
  1. Iniciar el flujo de datos (socket, kafka, etc).
  2. `spark-submit script.py`. Recuerdad incluir `ssc.awaitTermination()`.
  3. Comenzar a enviar datos.
  4. Finalizar con Ctrl-C.
- Desde iPython o notebooks:
  1. Iniciar el flujo de datos (socket, kafka, etc).
  2. Ejecutar el código (sin `awaitTermination()`).
  3. Comenzar a enviar datos.
  4. Finalizar con la función `ssc.stop()`.

Ventanas deslizantes en Spark Streaming

Spark Streaming permite realizar ventanas deslizantes a través de la función miembro `window()`.

- Dichas ventanas son sobre los RDDs creados anterioremente: agrega varios RDDs en una única ventana.
- Características:
  - Longitud (length). Nº de RDDs que puede contener.
  - Desplazamiento (shift). Cantidad de solapamiento entre ventanas (en nº de RDDs)
  - El desplazamiento debe ser múltiplo de la duración del batch y el tamaño múltiplo del desplazamiento.
  - Spark presenta operaciones para trabajar a nivel de ventana.

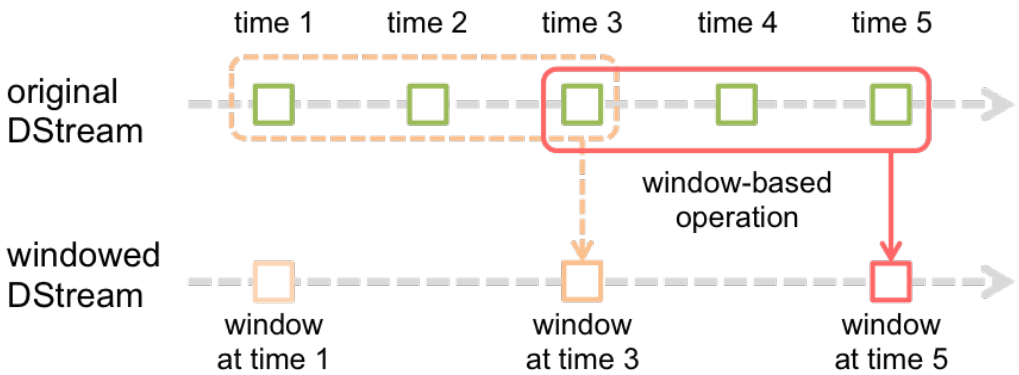


Figura: Ventana con tamaño 3, desplazamiento 2

Operaciones Stateful y Checkpointing

A veces es necesario realizar operaciones que actualicen un estado anterior del sistema. Esto se le conoce como operaciones *stateful*. Spark nos permite aplicar este proceso directamente a los `DStream` mediante la función `updateStateByKey()`. Ejemplo:

```
1  from pyspark import SparkContext
2  from pyspark.streaming import StreamingContext
3
4  sc = SparkContext()
5  ssc = StreamingContext(sc, 5)
6  ssc.checkpoint("checkpoint")
7
8  lines = ssc.socketTextStream("localhost", 9999)
9
10 def updateFunc(new_values, last_sum):
11 return sum(new_values) + (last_sum or 0)
12
13 running_counts = lines.flatMap(lambda line: line.split("
    ")).map(lambda word: (word,1)).updateStateByKey(
   updateFunc)
14 running_counts.pprint()
15
16 context.start()
17 context.awaitTermination()
18
```

Operaciones Stateful y Checkpointing

- Esta operación se asemeja a un `foldLeft()`, pero separando el estado por clave-valor y aplicando todos los procesos de Spark para tolerancia a fallos de manera transparente al usuario.
- Solo se debe activar el *checkpointing*: es un backup que se realiza del estado y metadatos a un almacenamiento tolerante a fallos (como HDFS).
- Se crean de manera periódica, **pero producen una sobrecarga de I/O**. Hay que elegir un balance entre tolerancia a fallos y rendimiento.

Lectura Recomendada

- Gerard Maas y Francois Garillot. **Stream Processing with Apache Spark: Mastering Structured Streaming and Spark Streaming**. O'Reilly Media, 2019. Capítulos del 16 al 22 y el 24.
- Documentación de Apache Spark Streaming: `https://spark.apache.org/docs/latest/streaming-programming-guide.html`
- Vídeos "Apache Spark Streaming with Python and PySpark": `https://learning.oreilly.com/videos/apache-spark-streaming/9781789808223/`
  - Pyspark setup tutorial
  - Pyspark basics (Transformations, Window)
  - Stateful transformations y checkpointing
  - Pyspark streaming with Apache Kafka